COMPUTER VISION COURSE · SESSION 7

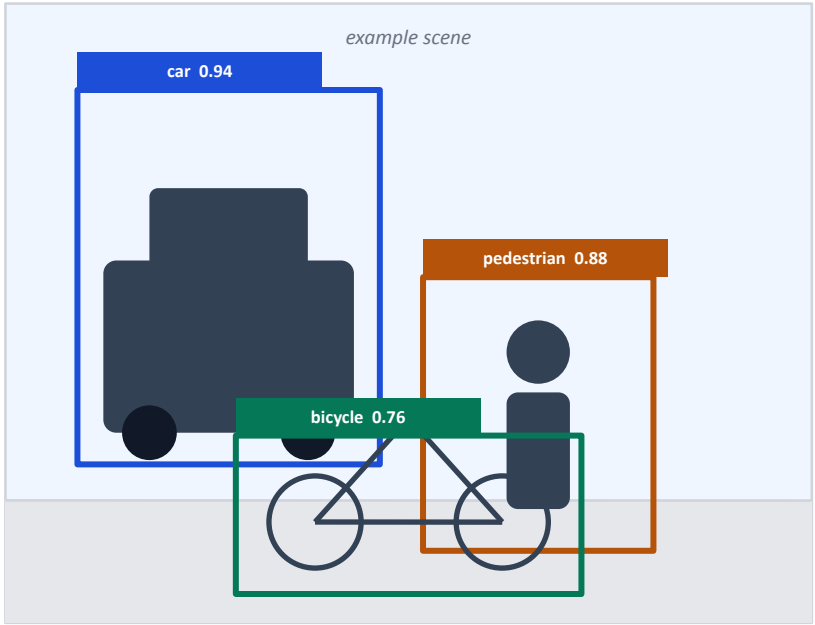
One-Stage Object Detectors & Evaluation Strategies

SSD and the full YOLO lineage (v1→v12), then a deep, illustrated tour of how we actually measure a detector's quality — IoU, Dice, PR curves, AP, mAP, and the COCO metrics suite.

CS231n-inspired · intuition first, then technical depth

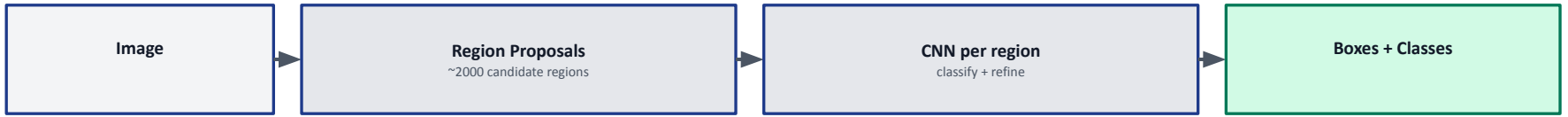
THE DETECTION PROBLEM : One image, an unknown number of objects

- Classification: 1 image ⟶ 1 label.
- Detection: 1 image ⟶ N objects, N unknown in advance.
- **For every object we must recover, jointly:**
  - Localization — a bounding box (x, y, w, h)
  - Classification — which category the box contains
  - Confidence — how sure the model is
- *This dual nature is exactly why detection needs its own architectures AND its own evaluation metrics — "accuracy" alone can't capture "right box, right class."*



Two-stage vs. one-stage detectors

Two-stage (propose, then classify) — R-CNN family, not today's focus:



One-stage (predict directly) — SSD, YOLO — today's focus:



- No separate proposal stage ⟶ one network, one pass, far fewer redundant computations ⟶ real-time speed.
- The trade-off (historically): one-stage detectors traded some accuracy — especially on small/crowded objects — for a large speed advantage. Closing that gap is the whole story of YOLOv2 ⟶ v12.

TODAY'S ROADMAP

1

SSD

Single Shot MultiBox Detector — multi-scale feature maps + default boxes

2

YOLOv1

“You Only Look Once” in full depth — grid cells, loss function, architecture

3

YOLOv2 → YOLOv12

The mainline lineage — what changed at each step, and why

4

Evaluating detectors

IoU, Dice, precision/recall, AP, mAP, COCO metrics — all with illustrations

THINK ABOUT IT *Before we dive in: why might predicting boxes directly (regression) be a harder learning problem than predicting a single class label?*

1

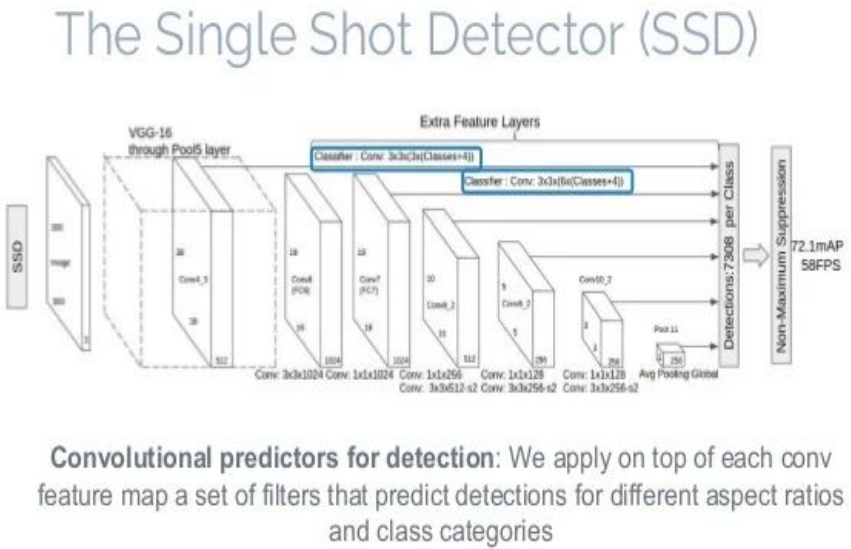
PART 1

SSD — Single Shot MultiBox Detector

Liu et al., 2016 (ECCV). The first widely-used one-stage detector to match two-stage accuracy while running in real time, by predicting from several feature-map resolutions at once.

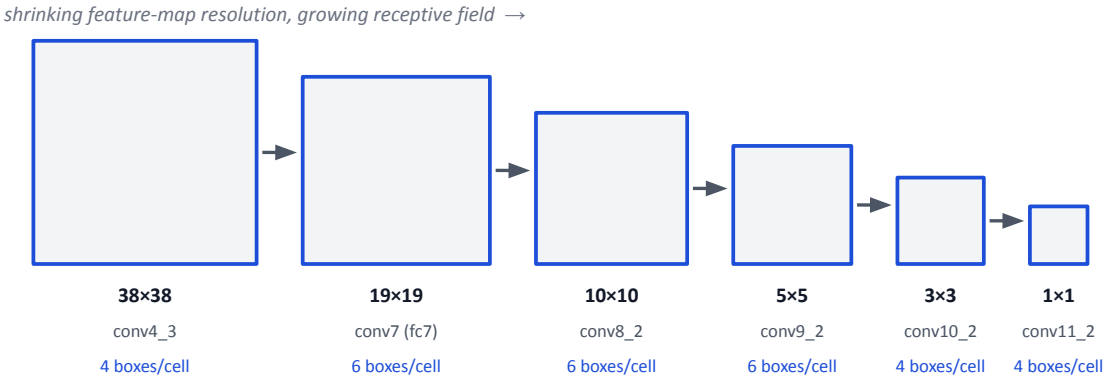
Why “single shot”?

- Two-stage detectors spend most of their time proposing regions, then re-running a classifier on each one.
- SSD's idea: skip the proposal stage entirely — discretize the space of possible boxes up front (“default boxes”), and let one CNN predict class scores + box offsets for every one of them in a single forward pass.
- **The key extra ingredient over a naive single-scale version:**
  - predict from multiple feature-map resolutions, not just the last layer — this is what lets SSD handle both large and small objects.



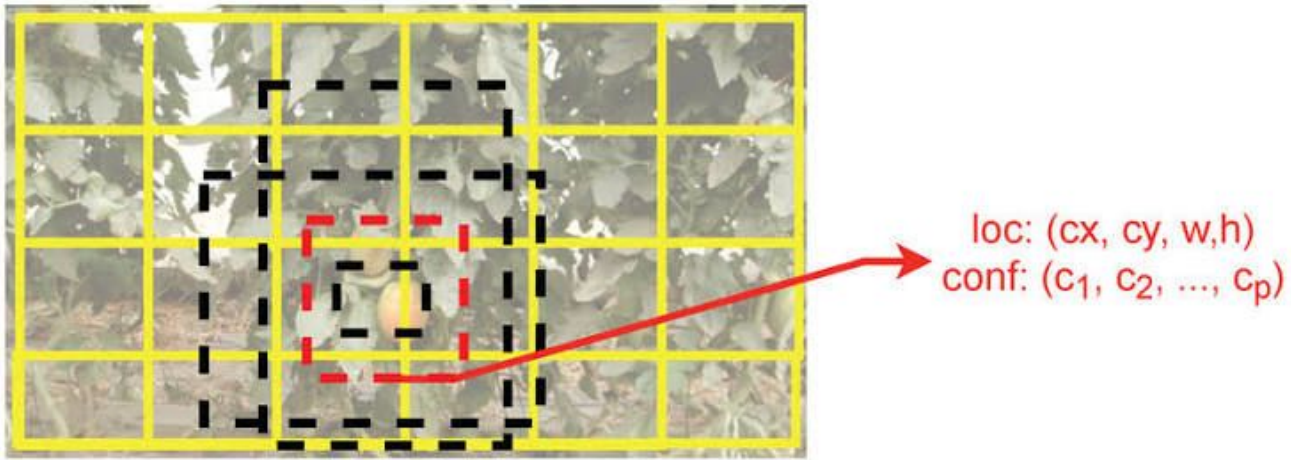
SSD · MULTI-SCALE PREDICTION

- Early, high-resolution maps (38×38) have small receptive fields → detect small objects. Late, low-resolution maps (1×1) have seen the whole image → detect large objects.
- Total predictions across all six maps: 8,732 default boxes per image, all scored in one pass.



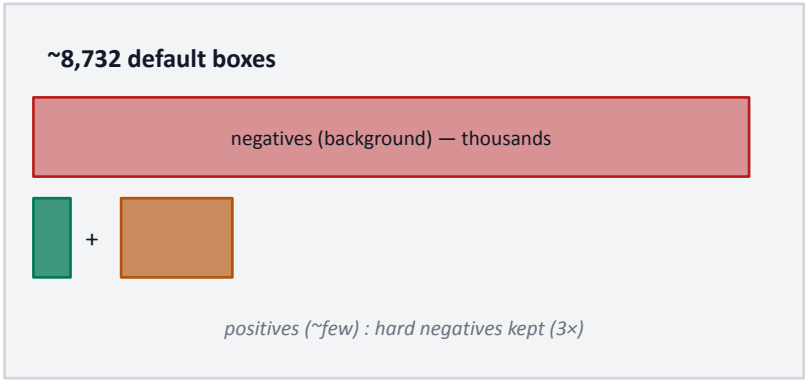
SSD · DEFAULT BOXES

- At every cell of every feature map, SSD centers several “default boxes” (anchor boxes) of different aspect ratios and scales.
- **Scale for feature map k (of m):**
  - $s_k = s_{min} + (s_{max} - s_{min})/(m-1) \cdot (k-1)$, $s_{min}=0.2$, $s_{max}=0.9$
- Aspect ratios used: {1, 2, 3, 1/2, 1/3}, plus one extra square box at scale $\sqrt{(s_k \cdot s_{k+1})}$ for ratio = 1.
- Network output per box: 4 offsets (Δcx, Δcy, Δw, Δh) + (C+1) class scores (C classes + background).



SSD · MATCHING & HARD NEGATIVE MINING

- **Matching strategy**
  - each ground-truth box is matched to its best-overlap default box, AND to every default box with Jaccard overlap (IoU) > 0.5
  - ⟶ many positive matches per object, not just one.
- **The imbalance problem**
  - a typical image has a handful of positive (matched) boxes out of 8,732 and overwhelmingly negative (background).
- **Hard negative mining**
  - rank negative boxes by confidence loss (the ones the model is most wrong about) and keep only enough negatives to reach a neg:pos ratio of 3:1.
  - stabilizes training and speeds up convergence versus using all negatives.



SSD · LOSS FUNCTION

$$L(x, c, l, g) = (1/N) \cdot (L_{conf}(x, c) + \alpha \cdot L_{loc}(x, l, g))$$

N = number of matched (positive) default boxes · α = 1 (balances the two terms)

- **L_loc — Smooth L1 loss**
  - between predicted (Δcx,Δcy,Δw,Δh) offsets and the encoded ground-truth offsets
  - computed ONLY over positive (matched) boxes
- **L_conf — Softmax loss**
  - multi-class confidence loss over (C+1) classes
  - computed over positives + the mined hard negatives

Detector	VOC07 mAP	Speed
SSD300	72.1%	58 FPS
SSD512	75.1%	—
Faster R-CNN	73.2%	7 FPS

SSD roughly matches two-stage Faster R-CNN accuracy at ~8× the speed, and beats YOLOv1 on both axes.

2

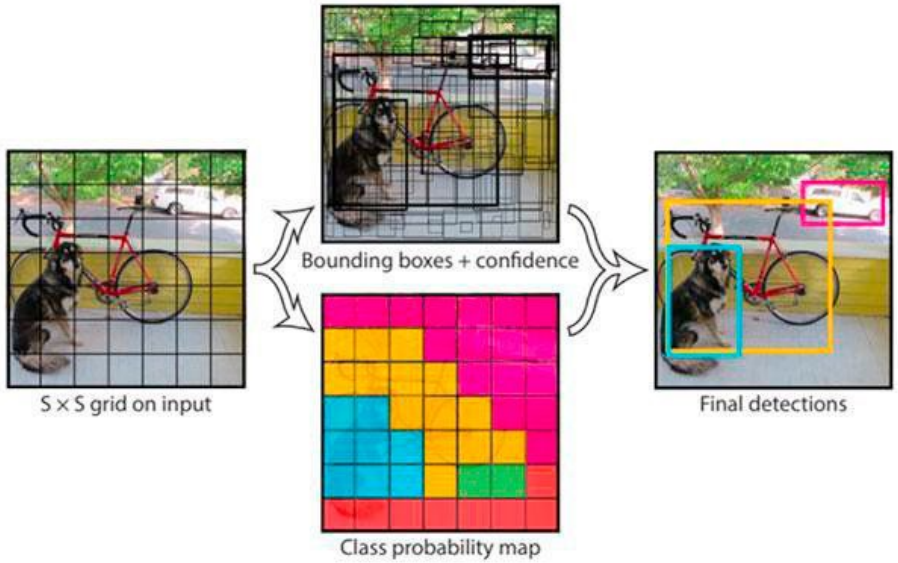
PART 2

YOLOv1 — You Only Look Once

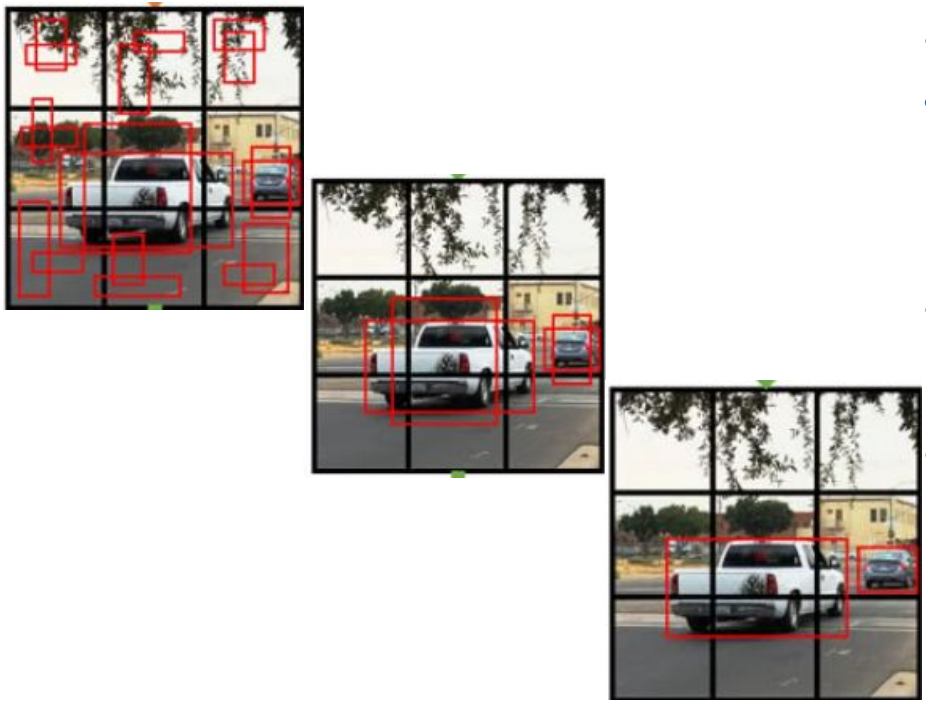
Redmon, Divvala, Girshick & Farhadi, 2016 (CVPR). Detection reframed as a single regression problem, from image pixels straight to box coordinates and class probabilities.

YOLOV1 · INTUITION

- SSD (and the R-CNN family) still think in terms of many candidate boxes scored independently.
- **YOLOv1's reframing:**
  - look at the WHOLE image once, and directly regress a fixed-size tensor that encodes every box, confidence, and class probability — no proposals, no sliding windows.
- Because it sees the entire image at once (not just local windows), YOLOv1 implicitly encodes global context about classes and their appearance — it makes far fewer background-as-object mistakes than region-proposal methods of its time.

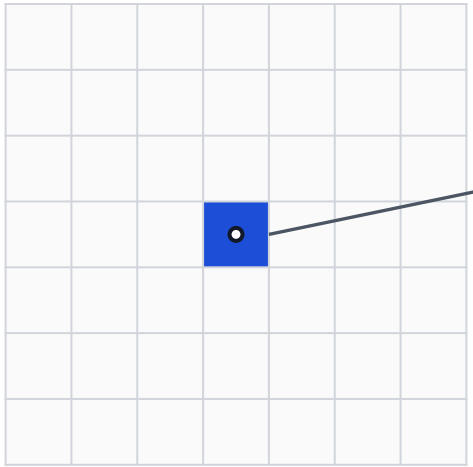


YOLOV1 · THE S×S GRID



- The input image is divided into an S×S grid (S=7).
- **Responsibility rule:**
  - a grid cell is “responsible” for detecting an object if and only if the object's ground-truth center falls inside that cell (the dashed box and white dot on the left).
- Each responsible cell predicts B=2 bounding boxes (candidates), plus one set of class probabilities for whatever object it's responsible for.
- Every other cell (no object center inside it) is trained to predict low confidence — “nothing here.”

YOLOV1 · WHAT EACH CELL PREDICTS





Box 1: x, y, w, h, confidence (5)

Box 2: x, y, w, h, confidence (5)

Class probabilities, C=20 (PASCAL VOC) (20)

one grid cell → 30 values

- $S = 7$: 7×7 grid cells
- $B = 2$: 2 box predictors per cell
- $C = 20$: 20 class probabilities per cell

Each cell predicts:

$$\underbrace{(x_1, y_1, w_1, h_1, \text{obj}_1)}_{\text{Box 1: 5}} + \underbrace{(x_2, y_2, w_2, h_2, \text{obj}_2)}_{\text{Box 2: 5}} + \underbrace{(P_1, \ldots, P_{20})}_{\text{classes: 20}}$$

Therefore:

$$5B + C = 5(2) + 20 = 30$$

and the complete output is:

$$\boxed{7 \times 7 \times 30 = 1470}$$

YOLOV1 · CONFIDENCE, PRECISELY DEFINED

$$\text{Box confidence} = \Pr(\text{Object}) \times \text{IOU}(\text{pred}, \text{truth})$$

→ 0 if no object exists in the cell; otherwise, how well the predicted box actually fits the object

$$\text{Class-specific confidence (test time)} = \Pr(\text{Class}_i \mid \text{Object}) \times \Pr(\text{Object}) \times \text{IOU} = \Pr(\text{Class}_i) \times \text{IOU}$$

→ encodes both "how likely is class i" AND "how good is this exact box" in one scalar per class, per box

- Class probabilities Pr(Class_i | Object) are predicted once per CELL (conditioned on there being an object), not once per box — both of the cell's boxes share the same class distribution.
- This shared-class-per-cell design is exactly the limitation we'll return to at the end of this section.

From YOLO outputs to class-specific confidence

Each bounding box predicts (x, y, w, h) and a confidence score (Pr(Object) × IoU).
We combine this with class probabilities to get a confidence score for each class.

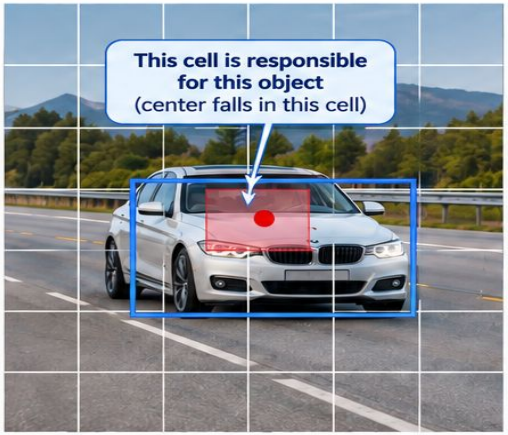


Image divided into S × S grid
(e.g., 7 × 7)

1 Box predictor output (for one box)

Box coordinates (4 values)
x, y, w, h
(center x, center y, width, height)

Confidence (1 value)
Pr(Object) × IoU
How likely this box contains an object and how well it overlaps (IoU) with the object.

2 Class probabilities (for this cell)
$Pr(Class_i \mid Object)$

P(Car | Object)	P(Dog | Object)	…	P(Cat | Object)	…
0.9	0.05		0.03	

3 Class-specific confidence (test time)

$$Pr(Class_i \mid Object) \times Pr(Object) \times IoU = Pr(Class_i) \times IoU$$

Car confidence	Dog confidence	…	Cat confidence	…
= 0.9 × 0.8 = **0.72**	= 0.05 × 0.8 = **0.04**		= 0.03 × 0.8 = **0.024**	

Example numbers
(for one predicted box)

x = 0.52 (relative to cell)
y = 0.48
w = 0.40
h = 0.30

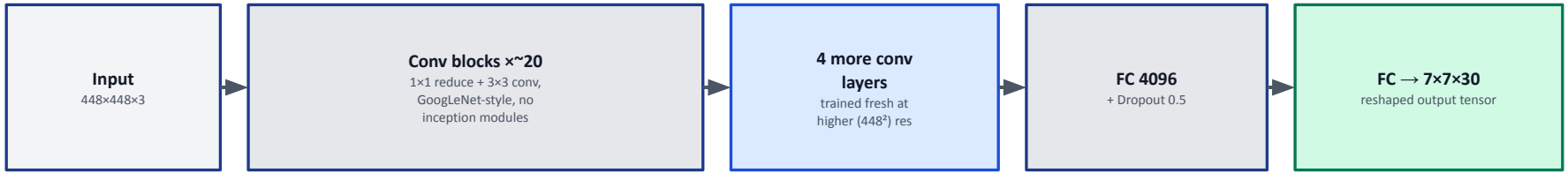
Confidence = 0.8
(= Pr(Object) × IoU)

We compute a confidence score for each class. During inference, we keep high-scoring boxes and apply Non-Maximum Suppression (NMS) to remove duplicates.

Key takeaways:

- Each bounding box predicts 5 values: x, y, w, h, confidence (Pr(Object) × IoU).
- Each cell predicts one set of class probabilities: $Pr(Class_i \mid Object)$.
- At test time, class-specific confidence = $Pr(Class_i \mid Object) \times Pr(Object) \times IoU = Pr(Class_i) \times IoU$.
- In YOLOv1, the class probabilities are shared by both boxes in the cell.

YOLOV1 · NETWORK ARCHITECTURE



- **Pretraining**
  - the first ~20 conv layers are pretrained as an ImageNet classifier at 224×224 (a callback to Session 3's backbone story).
- **Detection fine-tuning**
  - resolution is bumped to 448×448 (fine spatial detail matters more for localization), 4 conv layers + 2 FC layers are added, and the whole network is fine-tuned end to end.
- Activations: Leaky ReLU (α=0.1) everywhere except the final layer, which is linear.

YOLOv1 Loss Function (Sum of Squared Errors)

YOLOv1 optimizes a single loss that combines 4 terms: **localization** (box coordinates), **object confidence**, no-object confidence, and **class probabilities**.
Only one bounding box predictor per object is responsible for the prediction (the one with higher IoU to the ground truth).

$$\mathcal{L} = \lambda_{\text{coord}} \sum_{i=0}^{S^2-1} \sum_{j=0}^{B-1} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] + \sum_{i=0}^{S^2-1} \sum_{j=0}^{B-1} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2$$

(1) Localization loss (box coordinates) — (2) Object confidence loss (for responsible box)

$$+ \lambda_{noobj} \sum_{i=0}^{S^2-1} \sum_{j=0}^{B-1} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2-1} \mathbb{1}_{i}^{obj} \sum_{c=1}^{C} (p_i(c) - \hat{p}_i(c))^2$$

(3) No-object confidence loss (for boxes without object) — (4) Class probability loss (for cells with an object)

- S : grid size (e.g., 7)
- B : number of boxes per cell (e.g., 2)
- C : number of classes (e.g., 20)
- $\mathbb{1}_{ij}^{obj}$: 1 if j-th box in cell i is responsible for a ground truth object, else 0
- $\mathbb{1}_{ij}^{noobj}$: 1 if the box does not contain an object, else 0
- $\mathbb{1}_{i}^{obj}$: 1 if cell i contains an object, else 0
- λ_{coord} : typically 5 (higher weight for localization)
- λ_{noobj} : typically 0.5 (lower weight for no-object)

1 Localization loss (x, y, w, h)

Only for the responsible bounding box in cells that contain an object.



This cell is responsible (center falls here)

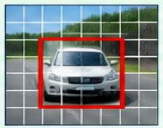
Predict:
- x (center x)
- y (center y)
- w (width)
- h (height)

- Penalize errors in box coordinates (x, y, w, h).
- Square root for w, h reduces the penalty for large boxes.
- **Weighted** by λ_{coood} (typically 5).

$$\lambda_{\text{coord}} \mathbb{1}_{ij}^{obj} \left[(x - \hat{x})^2 + (y - \hat{y})^2 + (\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2 \right]$$

2 Object confidence loss

Only for the responsible bounding box in cells that contain an object.



Predict:
$C = \Pr(\text{Object}) \times \text{IoU}$

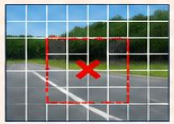
Target:
$\hat{C} = \text{IoU}$
(for responsible box)

- Encourages high confidence when a predicted box matches a ground truth.
- Confidence is trained to approximate the IoU with the ground truth (since Pr(Object) = 1 for responsible box).

$$\mathbb{1}_{i}^{obj} (C - \hat{C})^2$$

3 No-object confidence loss

For all boxes in cells that do not contain an object, and for non-responsible boxes in cells with an object.



Predict:
$C = \Pr(\text{Object}) \times \text{IoU}$

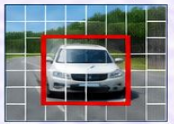
Target:
$\hat{C} = 0$
(no object)

- Penalizes confidence scores for boxes that do not contain any object.
- Prevents the model from predicting high confidence everywhere.
- **Weighted** by λ_{noobj} (typically **0.5**, so this term has lower weight).

$$\lambda_{noobj} \mathbb{1}_{ij}^{noobj} (C - 0)^2$$

4 Class probability loss

Only for cells that contain an object (one set of class probabilities per cell, shared by both boxes).



Predict:
$P(\text{class}_1), \ldots, P(\text{class}_C)$

Target:
one-hot vector
(e.g., [0, 0, 1, ..., 0] for "car")

- Penalizes error in class probabilities using squared error.
- Only computed for cells that contain an object ($\mathbb{1}_{i}^{obj}$).
- **Class probabilities** are **conditioned on** there being an object in the cell.

$$\mathbb{1}_{i}^{obj} \sum_{c=1}^{C} (p(c) - \hat{p}(c))^2$$

Key points:

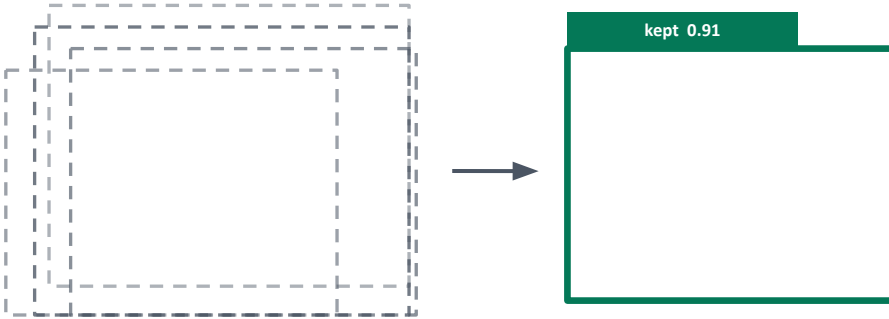
- Total loss is a weighted sum of 4 terms.
- Only one box predictor per object is "responsible" (the one with higher IoU to the ground truth).
- Confidence = Pr(Object) × IoU (single value predicted by the network, continuous between 0 and 1).
- Class probabilities are predicted per cell (not per box) and are conditioned on the presence of an object.
- $\lambda_{coord} = 5$ and $\lambda_{noobj} = 0.5$ are used to balance the different terms.

What does the model learn?

- Predict accurate box coordinates for objects.
- Predict high confidence for true objects and low confidence otherwise.
- Predict the correct class for the object in each cell.
- All in a single forward pass, end-to-end.

YOLOV1 · NON-MAX SUPPRESSION

- 7×7 cells × 2 boxes = 98 candidate boxes per image, even though most objects should produce just one final detection.
- **Non-max suppression (NMS), applied after the network's forward pass:**
  - sort remaining boxes by confidence
  - keep the highest-confidence box
  - suppress (discard) any other box that overlaps it above an IoU threshold
  - repeat until no boxes remain
- *NMS is not learned — it's a fixed post-processing step used by nearly every detector in this session, one-stage or two-stage.*



before NMS: several overlapping boxes

after NMS: one box per object

STRENGTHS & LIMITATIONS

STRENGTHS

- Extremely fast (45 FPS base model, 155 FPS for “Fast YOLO”)
- Global reasoning over the whole image → far fewer background-as-object false positives than region-proposal methods
- Generalizes well to new domains (the paper shows strong results on artwork/paintings, not just photos)
- Simple, unified, end-to-end trainable

LIMITATIONS

- Only 1 class per cell + 2 boxes per cell → struggles when multiple small objects share a cell (e.g. a flock of birds)
- Coarse spatial grid + heavy downsampling → imprecise localization, especially for small objects
- Struggles to generalize to unusual aspect ratios or configurations not well represented in training boxes
- Loss treats a large and a small box's coordinate error somewhat similarly despite the sqrt fix — localization error is the single biggest error source in the paper's own analysis

3

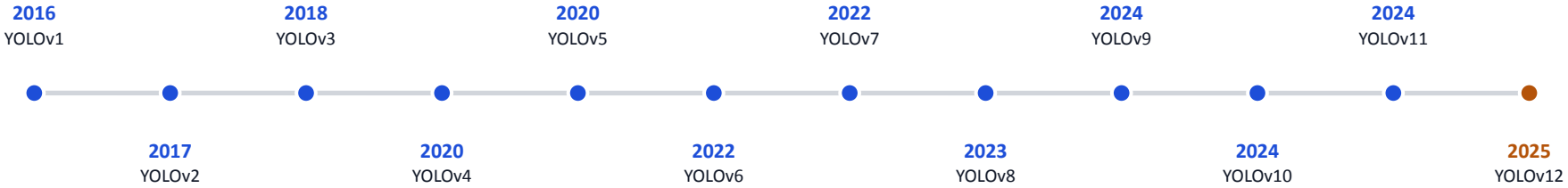
PART 3

YOLOv2 → YOLOv12 — the mainline lineage

We already covered YOLOv1 in full depth. From here, one slide per NOTABLE release — a version that introduced a genuinely new idea, not just a refinement. (Ultralytics-lineage forks like YOLOX, YOLOR and PP-YOLO stay out of scope today.)

The mainline timeline at a glance

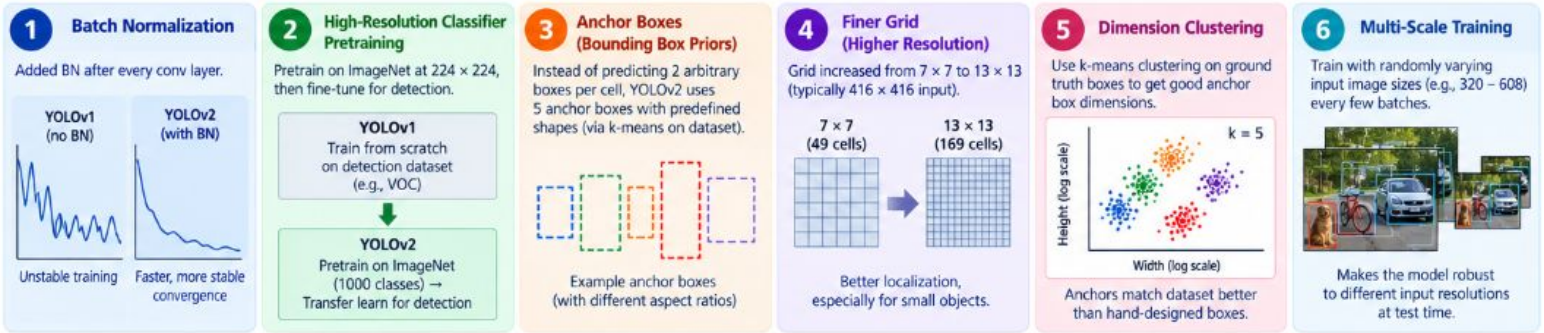
- Three eras, roughly: the Darknet era (v1–v4, original authors), the Ultralytics/PyTorch engineering era (v5, v8, v11), and a burst of parallel research groups racing on efficiency (v6, v7, v9, v10, v12).
- Next up, a dedicated slide for each NOTABLE version — v2, v3, v4, v5, v7, v8, v10, v12. v6, v9, and v11 still get a row in the closing summary table; each refines an idea a neighboring version already introduced rather than adding a new one.



YOLOv2 (YOLO9000)

Darknet-19 · Anchor-based (new)

- Anchor boxes via k-means clustering on training-set boxes (“dimension clusters”), replacing YOLOv1's free-form box regression.
- Batch normalization on every conv layer (+2% mAP) and a high-resolution (448*448) classifier pretraining stage.
- A passthrough (skip) layer concatenates an earlier, higher-resolution feature map in — a first small step toward multi-scale features.
- Multi-scale training: input size randomly changed every 10 batches, so one network works across resolutions.
- YOLO9000: joint classification + detection training over a WordTree label hierarchy — detects >9,000 categories with only ~200 having detection labels.



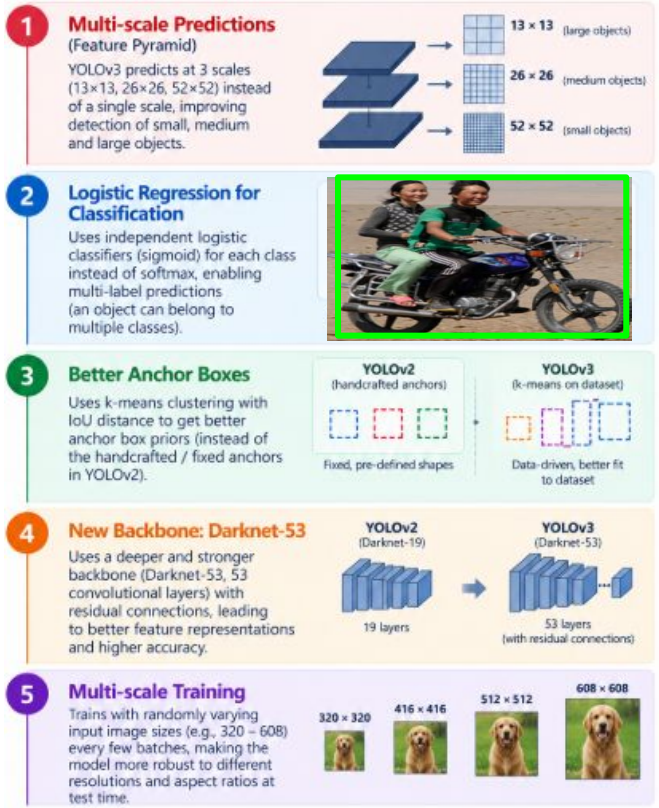
76.8% mAP VOC07 @ 67 FPS · 21.6% mAP COCO (a much stricter benchmark)

YOLO LINEAGE · 2018

YOLOv3

Darknet-53 (residual) · Anchor-based, 3 scales

- Darknet-53: a deeper, residual (ResNet-style) backbone — a direct callback to Session 3's architecture story — far better features than v2's Darknet-19.
- Predicts at 3 different scales (via upsampling + concatenation with earlier layers, FPN-like) with 3 anchors each — 9 total, still k-means clustered.
- Independent per-class logistic classifiers + binary cross-entropy, replacing softmax — enables multi-label predictions (e.g. “person” AND “woman”).

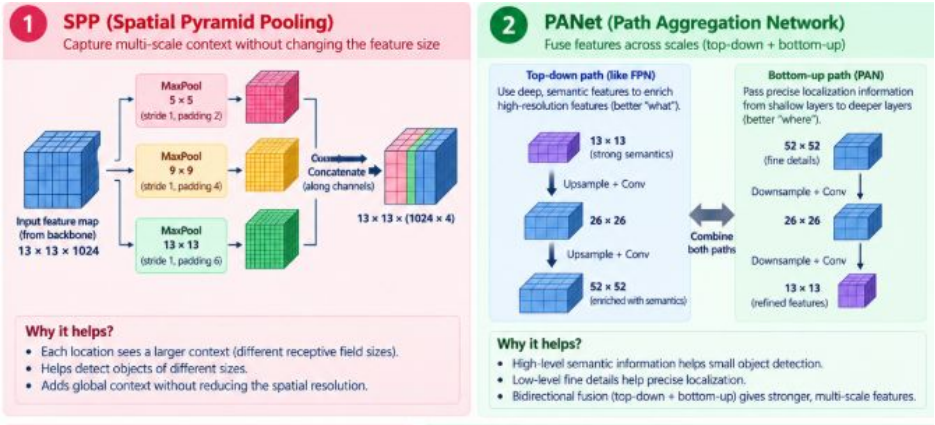


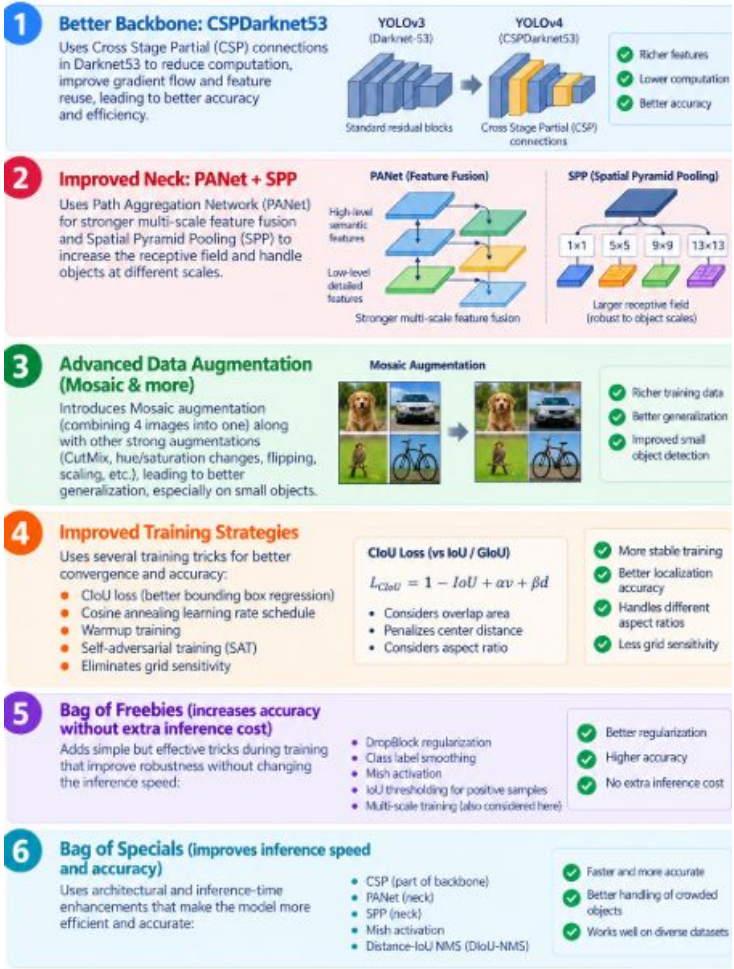
57.9% AP50 on COCO — on par with SSD variants and RetinaNet, at 3–4× the speed

YOLOv4

CSPDarknet-53 · Anchor-based

- CSPDarknet-53: Cross-Stage-Partial connections cut computation while preserving accuracy.
- “Bag of Freebies” (free at inference): Mosaic augmentation (stitches 4 training images together), CutMix, label smoothing, DropBlock.
- “Bag of Specials” (small inference cost): SPP block (enlarges receptive field), PANet neck (richer feature fusion than v3's FPN), Mish activation.
- CIoU loss for box regression — penalizes overlap, center distance, AND aspect-ratio mismatch together (more on this in Part 4).

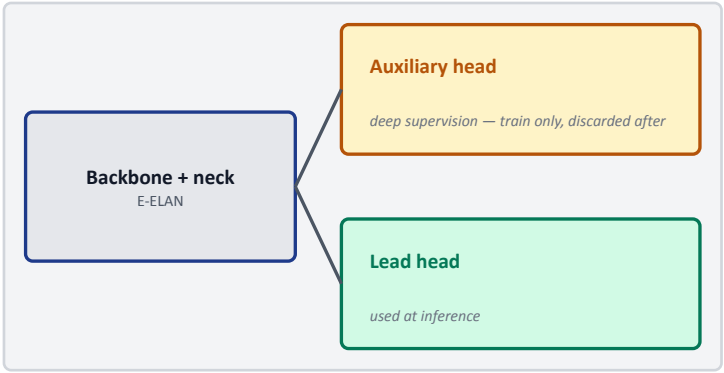


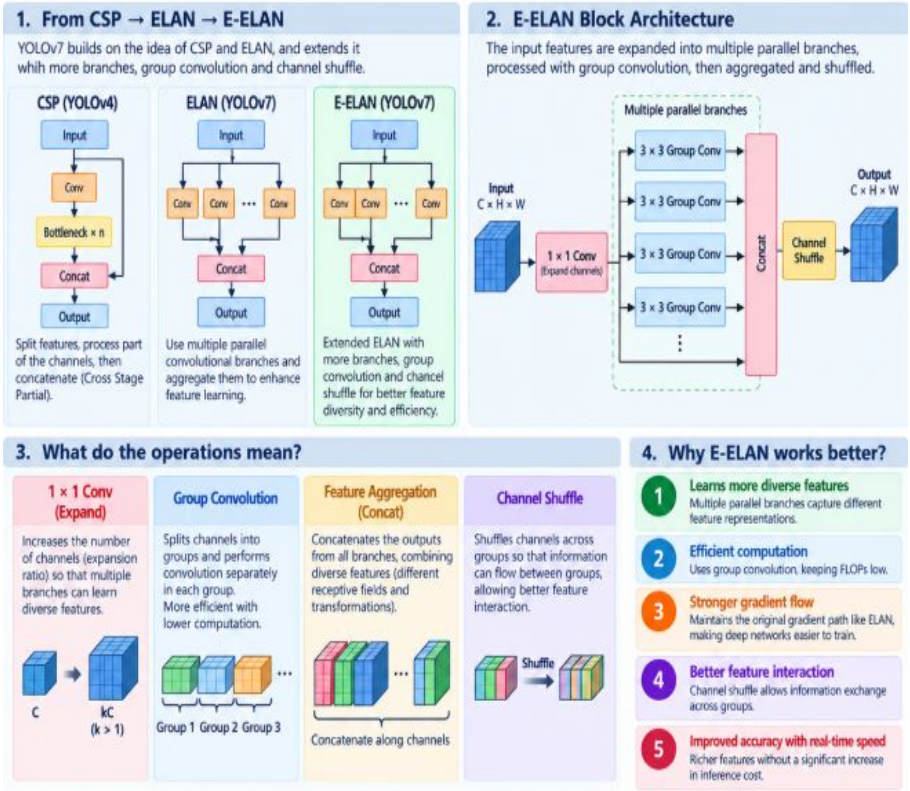


YOLOv7

E-ELAN · Anchor-based

- Same lineage as YOLOv4 (Wang, Bochkovskiy, Liao). E-ELAN controls gradient path length so very deep networks stay efficient to train.
- “Trainable bag-of-freebies”: a coarse-to-fine auxiliary head (deep supervision at a middle layer) guides training; only the lead head is used at inference — zero extra deployment cost.
- Planned re-parameterization, applied selectively based on gradient-flow analysis (a refinement of YOLOv6's reparam idea).

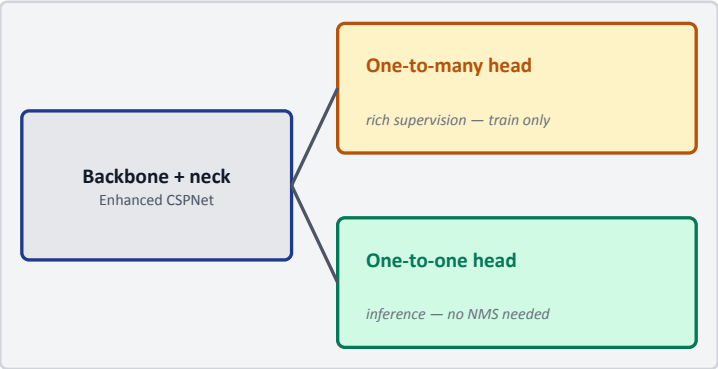




YOLOv10

Enhanced CSPNet · Anchor-free, NMS-free

- Tsinghua University (THU-MIG). Headline change: consistent dual assignment — a one-to-many head (rich supervision, as in every earlier YOLO) trained IN PARALLEL with a one-to-one head.
- At inference, only the one-to-one head runs — no NMS post-processing step needed AT ALL, the first version to remove something every detector since SSD has needed.
- Holistic efficiency redesign: lightweight classification head, spatial-channel decoupled downsampling, a partial self-attention (PSA) module for cheap global context.



WHY IT MATTERED

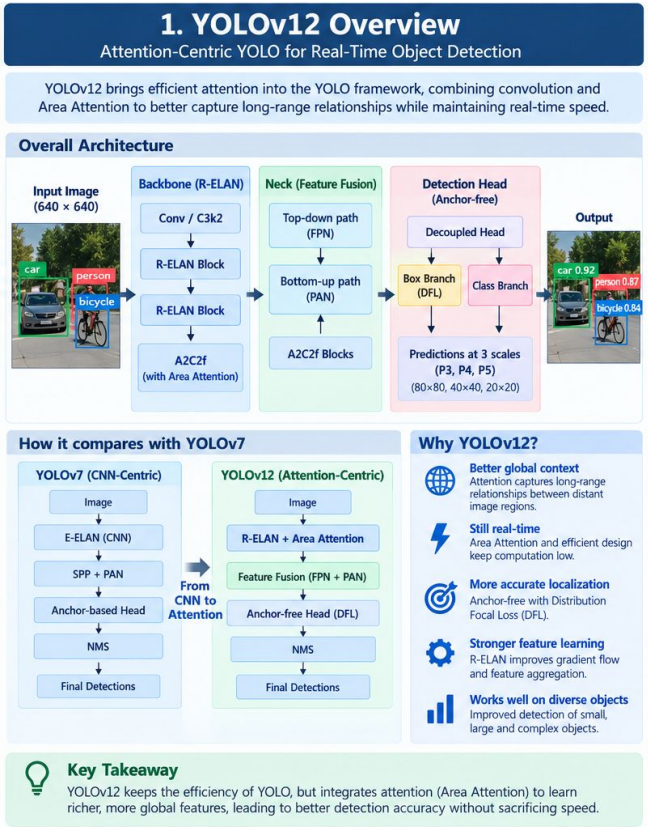
First mainline version to remove NMS entirely — a fixed, non-learned post-processing step every one-stage detector this session has needed, until now.

YOLOv10-B: ~46% lower latency, 25% fewer params than YOLOv9-C at comparable accuracy

YOLOv12

Area Attention + R-ELAN · Anchor-free, attention-centric

- Tian, Ye & Doermann, 2025 (NeurIPS). The first mainline version built around attention (transformer-style) rather than pure convolution — earlier attention mechanisms were considered too slow for real-time detection.
- Area Attention (A^2): partitions the feature map into large regions (“areas”) instead of attending globally — keeps a large effective receptive field while cutting attention's usual quadratic cost.
- R-ELAN: residual Efficient Layer Aggregation Network — block-level residual connections that stabilize training of an attention-centric architecture at depth.



YOLOv12

Anchor-Free Detection in YOLOv12

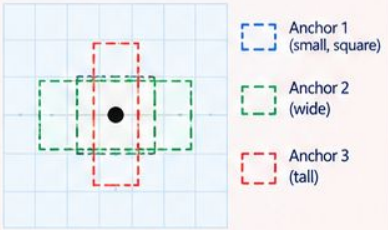
From predefined boxes to point-based prediction

Simpler.
More flexible.
More accurate.
Real-time.

1. Motivation: Why move beyond anchors?

Anchor-based (e.g., YOLOv7)

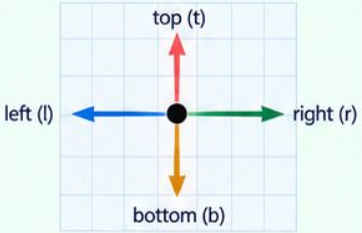
Predefine multiple anchor boxes at each location. Predict offsets relative to each anchor.



- Need to design anchor sizes and ratios
- Complex matching strategy
- Many redundant predictions
- Less flexible for unusual shapes

Anchor-free (YOLOv12)

No predefined boxes.
Each feature-map location is a point.
Predict distances to the 4 sides.

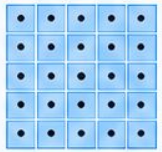


- No anchor boxes
- Simpler and more flexible
- Better for diverse object shapes
- Fewer hyperparameters

2. How it works: Point-based box prediction

1 Feature-map points

The image is converted to multi-scale feature maps (P3, P4, P5). Each cell (center point) is a candidate location.



P3 (80 × 80)
stride = 8
(6400 points)

P4 (40 × 40)
stride = 16
(1600 points)

P5 (20 × 20)
stride = 32
(400 points)

Total: **8400 points (for 640 × 640 input)**

2 Predict four distances

For each point, the network predicts the distances to the left, top, right and bottom boundaries of the object.



3 Convert to bounding box

$x_1 = x - l \quad y_1 = y - t$

$x_2 = x + r \quad y_2 = y + b$

$\text{Box} = (x_1, y_1, x_2, y_2)$



YOLOv12

3. Distribution Focal Loss (DFL) for precise localization

Instead of predicting a single number for each distance (l, t, r, b), YOLOv12 predicts a discrete distribution over multiple bins (reg_max = 16). The final distance is the expectation of this distribution.

Single distance prediction (e.g., left distance *l*)



Expected value:

$$l = \sum_i p_i \cdot i$$

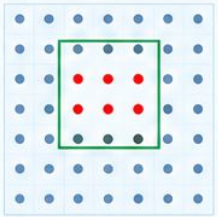
Output per point

- 4 distances × 16 bins = 64 regression values
- + class probabilities (e.g., 80 for COCO)

Total per point (COCO): 64 + 80 = 144 values

4. Training: Assigning points to ground truth

YOLOv12 uses Task-Aligned Assignment to select positive points.



Ground-truth box

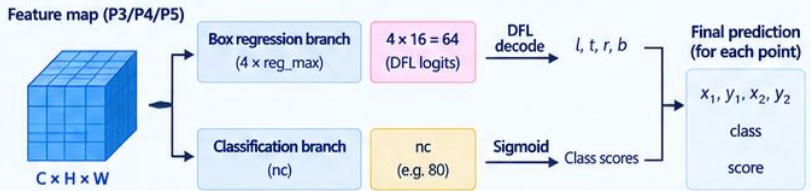
Positive points (selected by task alignment)

Negative points (background)

Task alignment considers:

- Classification score (how confident is the class?)
- Localization quality (IoU)
- Only the best points inside/near the object are assigned as positive.

5. Detection head in YOLOv12 (anchor-free, decoupled)



6. Inference: From predictions to final detections

After decoding boxes and class scores, standard post-processing (NMS) is applied.



Key advantages of anchor-free in YOLOv12

- No predefined anchor boxes
- More flexible for different object shapes
- DFL provides more precise localization
- Simpler design and training
- Works well for small and large objects
- Keeps real-time performance

Takeaway

YOLOv12 treats each feature-map location as a point and directly predicts the distances to the four sides of the object using a distribution-based approach (DFL), removing the need for predefined anchor boxes and enabling more flexible, accurate, and real-time object detection.

"A simpler representation for a more powerful detector."

THE MAINLINE LINEAGE, SIDE BY SIDE

Eleven versions, one table — ★ marks the ones we deep-dived

Version	Year	Backbone	Anchors	Headline idea
YOLOv1 ★	2016	Custom (24 conv)	None	Detection as one regression problem
YOLOv2 ★	2017	Darknet-19	Yes (k-means)	Anchors + batch norm + multi-scale training
YOLOv3 ★	2018	Darknet-53	Yes	Residual backbone + 3-scale prediction
YOLOv4 ★	2020	CSPDarknet-53	Yes	Bag of Freebies/Specials + CIoU loss
YOLOv5 ★	2020	CSPDarknet (PyTorch)	Yes	Engineering & deployment, not a paper
YOLOv6	2022	EfficientRep	No (default)	Re-parameterization for industrial deployment
YOLOv7 ★	2022	E-ELAN	Yes	Trainable bag-of-freebies, auxiliary head
YOLOv8 ★	2023	C2f	No	Fully anchor-free + unified multi-task framework
YOLOv9	2024	GELAN	No	Programmable Gradient Information (PGI)
YOLOv10 ★	2024	Enhanced CSPNet	No	NMS-free (dual label assignment)
YOLOv11	2024	C3k2 + C2PSA	No	Refinement pass + position-sensitive attention
YOLOv12 ★	2025	Area Attn + R-ELAN	No	Attention-centric, real-time budget

EXERCISE — Spot the pattern

Scan the Anchors column top to bottom. Anchors go from absent (v1) to present (v2–v7) to absent again (v8–v12). What does each transition tell you about the trade-off anchors are making?

4

PART 4

Evaluating object detectors

Every architecture we just covered reports a headline number — mAP, AP50, AP@[.5:.95]. This part is what those numbers actually mean, built up illustration by illustration from first principles.

“Accuracy” isn't well-defined for detection

- Classification accuracy is simple: predicted label == true label, count the fraction correct.
- **Detection has no such single yes/no. For every prediction we must ask two questions at once:**
  - Is the BOX right? (localization — a continuous, not binary, question)
  - Is the CLASS right? (classification — conditional on the box being acceptable)

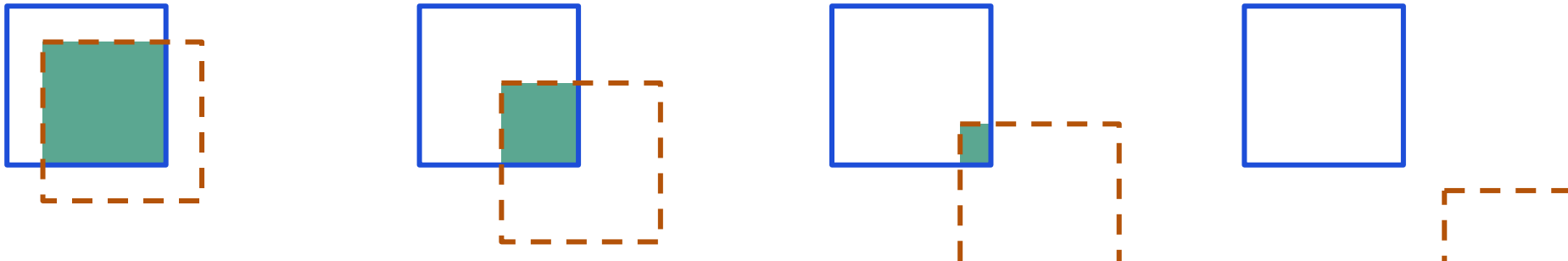
- **And unlike classification, a detector can fail in three distinct ways for one object:**
  - wrong box, right class · right box, wrong class · no box at all (missed) — plus a fourth failure with no classification analogue: a confident box where nothing exists.

THINK ABOUT IT *Which of those four failure types would worry you most in an ADAS perception stack — and does the answer depend on the object class (pedestrian vs. traffic cone)?*

INTERSECTION OVER UNION (IOU)

$$IoU(A, B) = area(A \cap B) / area(A \cup B)$$

intersection area over union area — always between 0 (no overlap) and 1 (identical boxes)



IoU = 0.82
near-perfect overlap

IoU = 0.41
partial overlap

IoU = 0.13
barely touching

IoU = 0.00
no overlap at all

- Cyan = ground truth, amber (dashed) = prediction, green fill = the intersection region. This is exactly the Jaccard overlap SSD used for box matching in Part 1.

True Positive, False Positive, False Negative

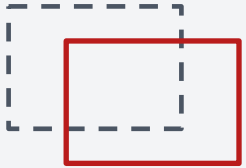
- Pick an IoU threshold (commonly 0.5). Every prediction is then scored against the ground truth:

True Positive (TP)



predicted box has IoU ≥ threshold with a real object, correct class

False Positive (FP)



predicted box has IoU < threshold with any object (or duplicates an already-matched box)

False Negative (FN)



a real object with no matching prediction at all — missed entirely

- Notice what's missing: there is no “true negative” in detection — there's no meaningful way to count all the boxes that were correctly NOT drawn. This is exactly why detection uses precision/recall/AP instead of plain accuracy.

PRECISION & RECALL, FOR DETECTION

Precision = TP / (TP + FP)

"of everything I predicted, how much was correct?"

Recall = TP / (TP + FN)

"of everything that was really there, how much did I find?"

- Both depend on the IoU threshold from the previous slide — change the threshold, and TP/FP/FN counts (and therefore precision and recall) all change with it.
- A detector can trivially get perfect recall (predict boxes everywhere) at terrible precision, or perfect precision (predict only when extremely confident) at terrible recall — neither number alone tells the full story.
- This tension — not one number, but a curve traced out as confidence threshold changes — is exactly what the next few slides build.

F1 SCORE

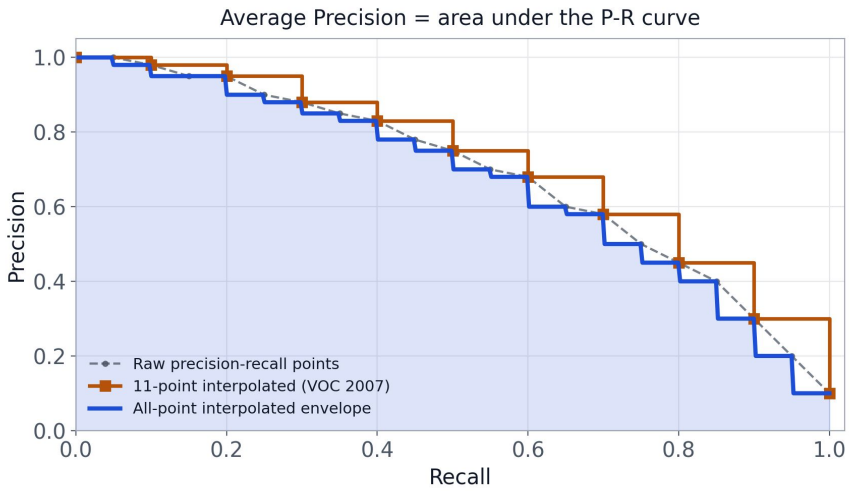
$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$$

the harmonic mean of precision and recall, AT ONE FIXED confidence threshold — unlike AP, which sweeps every threshold.

- Useful when you must ship ONE operating point (one confidence cutoff), not a whole curve — AP answers “how good could this model be”, F1 answers “how good is it AT THIS SETTING.”

THE PRECISION-RECALL CURVE

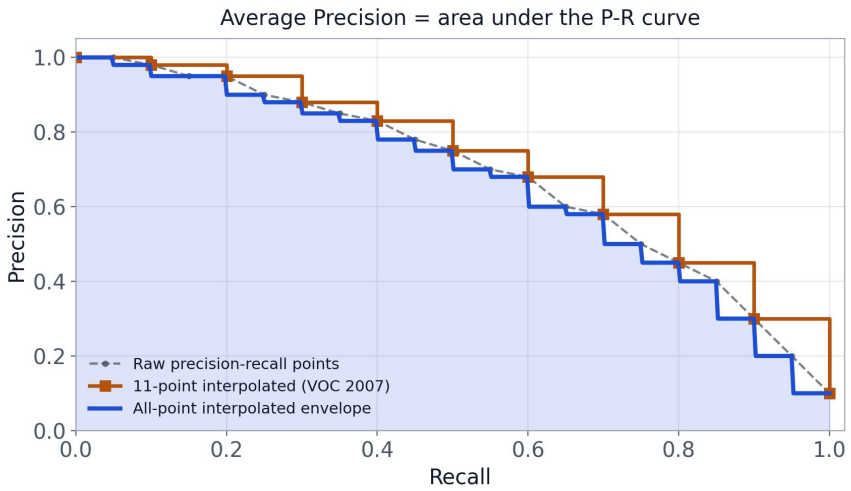
- Sort every prediction (across the whole test set, one class at a time) by confidence, highest first.
- Walk down the ranked list one prediction at a time; after each one, recompute precision and recall using everything seen so far.
- Plot precision (y) against recall (x) — that traces the gray zig-zag curve.
- High-confidence predictions tend to be correct → precision starts near 1. As we accept lower-confidence predictions, recall climbs but precision typically falls.



Gray = raw curve. Amber = 11-point interpolation (PASCAL VOC 2007). Blue = all-point (modern) interpolation.

AVERAGE PRECISION (AP)

- **AP = the area under the (interpolated) precision-recall curve, for one class.**
- **11-point interpolation (PASCAL VOC 2007)**
  - average interpolated precision at 11 fixed recall levels: 0, 0.1, 0.2, …, 1.0
- **All-point / monotonic interpolation (VOC 2010+, and the basis for COCO)**
  - exact area under the monotonically-decreasing envelope — no coarse 11-point approximation
- In both cases, “interpolated” precision at recall r = the maximum precision seen at any recall ≥ r — this removes the wiggles caused by ranking noise.



AP · WORKED EXAMPLE (1/2)

- Class = “car”. 5 ground-truth car boxes exist in the test set. A detector fires 5 detections; rank them by confidence and mark each TP/FP using IoU ≥ 0.5 (Part 4's own matching rule).

Rank	Confidence	TP / FP	Cum. TP	Cum. FP	Precision = TP/(TP+FP)	Recall = TP/5
1	0.95	TP	1	0	1.00	0.20
2	0.90	TP	2	0	1.00	0.40
3	0.85	FP	2	1	0.67	0.40
4	0.80	TP	3	1	0.75	0.60
5	0.70	FP	3	2	0.60	0.60

- Only 3 of the 5 detections are true positives, so recall tops out at 3/5 = 0.60 no matter how far down the ranked list we go — 2 ground-truth cars are simply never found.

THINK ABOUT IT *Precision actually goes UP between rank 3 and rank 4 (0.67 → 0.75), right after an FP. Walk through the arithmetic — why does that happen?*

AP · WORKED EXAMPLE (2/2)

1. Our example (ranked detections)

Rank	Confidence	TP / FP	Cumulative TP	Cumulative FP	Precision (TP/(TP+FP))	Recall (TP/5)
1	0.95	TP	1	0	1.00	0.20
2	0.90	TP	2	0	1.00	0.40
3	0.85	FP	2	1	0.67	0.40
4	0.80	TP	3	1	0.75	0.60
5	0.70	FP	3	2	0.60	0.60

There are 5 ground-truth cars, but **only 3 are detected correctly.**
So the **maximum recall achieved is 3/5 = 0.60.**

2. Interpolated precision values (VOC 2007)

For each recall level r, we take the **maximum precision** observed for any recall ≥ r:

$$p_{\text{interp}}(r) = \max_{\tilde{r} \geq r} p(\tilde{r})$$

Recall level r	Interpolated precision	Why?
0.0	1.00	highest precision is 1.00
0.1	1.00	highest precision is 1.00
0.2	1.00	highest precision is 1.00
0.3	1.00	highest precision is 1.00
0.4	0.75	max precision for r ≥ 0.4 is 0.75
0.5	0.75	max precision for r ≥ 0.5 is 0.75
0.6	0.75	max precision for r ≥ 0.6 is 0.75
0.7	0.00	no predictions with recall ≥ 0.7
0.8	0.00	no predictions with recall ≥ 0.8
0.9	0.00	no predictions with recall ≥ 0.9
1.0	0.00	no predictions with recall ≥ 1.0

4. Precision–Recall curve for the example



$$AP_{11} = (1/11) \cdot \Sigma\, p(r)$$

AP = (5×1.00 + 2×0.75 + 4×0.00) / 11 = 6.50 / 11

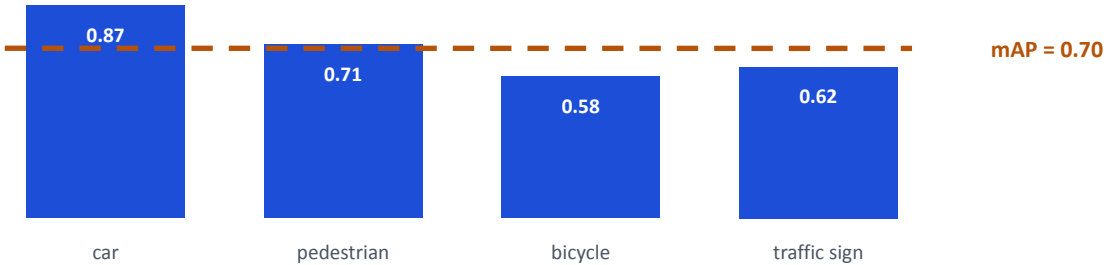
AP_car (11-point) = 0.591

MEAN AVERAGE PRECISION (MAP)

$$\text{mAP} = (1/N_\text{classes}) \cdot \Sigma(\text{class } c)\ AP_c$$

compute AP separately per class (car, pedestrian, bicycle, …), then average across all classes

- Averaging per class (rather than pooling all predictions together) stops common classes from drowning out rare ones — the same imbalance concern we saw in SSD's hard negative mining, in a different guise.



DICE IOU

- **Dice(A, B) = 2 · |A ∩ B| / (|A| + |B|)**
  - same ingredients as IoU, but the intersection is counted twice and divided by the sum of the two areas instead of their union.
- Algebraically related to IoU: Dice = 2·IoU / (1 + IoU) — Dice is always ≥ IoU for the same overlap (see chart).
- **Where each is used:**
  - IoU dominates object DETECTION (box matching, mAP, NMS)
  - Dice dominates pixel-wise SEGMENTATION, especially medical imaging — it weights the (often small) foreground region more forgivingly than IoU does.



EVALUATION, AT A GLANCE

Metric	What it answers	Where it shows up
IoU	How well do two boxes overlap?	Matching, NMS, TP/FP/FN, Dice
Precision / Recall	Correct-of-predicted? Found-of-real?	Building block for AP and F1
PR curve	How does the trade-off move with confidence?	Visual basis for AP
AP	Area under one class's PR curve	Per-class detector quality, one number
mAP	AP averaged over all classes	The number most papers lead with
COCO AP@[.5:.95]	mAP averaged over 10 IoU thresholds	Today's de facto standard benchmark
AP50 / AP75 / APS,M,L	AP at one threshold / by object size	Diagnosing where a model struggles
Dice	Overlap, weighted toward small regions	Segmentation (esp. medical imaging)
GIoU / DIoU / CIoU	Overlap-aware regression LOSS, not just a metric	Training signal (YOLOv4+)
FPS / latency	How fast, on what hardware?	The other half of every trade-off in Part 3
F1	Precision/recall balance at one threshold	Single deployed operating point

REFERENCES

- Liu, W. et al. (2016). SSD: Single Shot MultiBox Detector. ECCV. arXiv:1512.02325
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
- Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, Faster, Stronger. CVPR.
- Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934
- Li, C. et al. (2022). YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. arXiv:2209.02976
- Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2022). YOLOv7: Trainable Bag-of-Freebies. CVPR 2023.
- Wang, C.-Y., & Liao, H.-Y. M. (2024). YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. arXiv:2402.13616
- Wang, A. et al. (2024). YOLOv10: Real-Time End-to-End Object Detection. arXiv:2405.14458
- Tian, Y., Ye, Q., & Doermann, D. (2025). YOLOv12: Attention-Centric Real-Time Object Detectors. arXiv:2502.12524 (NeurIPS 2025)
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV. — source of the AP@[.5:.95] evaluation protocol
- Everingham, M. et al. (2010). The PASCAL Visual Object Classes (VOC) Challenge. IJCV. — source of the 11-point AP interpolation
- Zheng, Z. et al. (2020). Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. AAAI. — DIoU / CIoU
- Ultralytics (2020–2024). YOLOv5, YOLOv8, YOLO11 — open-source releases and documentation (docs.ultralytics.com).